

PROG5121 POE

ST10526600 K. KEKANA

Kate Kekana

Login class

```
import java.util.regex.Matcher;
```

```
import java.util.regex.Pattern;
```

```
public class LOGIN {
```

```
    private String registeredUsername;
```

```
    private String registeredPassword;
```

```
    private String firstName;
```

```
    private String lastName;
```

```
    private String registeredCellNumber;
```

```
    // Setters for names since they are used in the welcome messaging
```

```
    public void setNames(String firstName, String lastName) {
```

```
        this.firstName = firstName;
```

```
        this.lastName = lastName;
```

```
    }
```

```
    // Check if Username contains an underscore and is no more than 5 characters long
```

```
    public boolean checkUserName(String username) {
```

```
        if (username == null) return false;
```

```
        return username.contains("_") && username.length() <= 5;
```

```
    }
```

```
    // Check Password Complexity
```

```
    public boolean checkPasswordComplexity(String password) {
```

```
        if (password == null || password.length() < 8) return false;
```

```
        boolean hasUpper = false;
```

```
        boolean hasDigit = false;
```

```

boolean hasSpecial = false;

// Common regex pattern for special characters
Pattern special = Pattern.compile("[!@#$%^&*()_+\\-=\\[\\]\\{};':\\\",.<>/?~`|\\\\\\\\]");

for (char c : password.toCharArray()) {
    if (Character.isUpperCase(c)) hasUpper = true;
    if (Character.isDigit(c)) hasDigit = true;
}

Matcher hasSpecialMatcher = special.matcher(password);
if (hasSpecialMatcher.find()) hasSpecial = true;

return hasUpper && hasDigit && hasSpecial;
}

// Regular Expression-based cell phone checker for international code (+27 or 27) and
matching constraints
public boolean checkCellPhoneNumber(String cellNumber) {
    if (cellNumber == null) return false;

    // Matches +27 or 27 followed by 7 to 9 digits depending on exact localized interpretation
    String regex = "^((\\+27|27)\\d{7,9})$";

    Pattern pattern = Pattern.compile(regex);
    Matcher matcher = pattern.matcher(cellNumber);

    return matcher.matches();
}

// Registers the user and returns appropriate status messages
public String registerUser(String username, String password, String cellNumber) {
    if (!checkUserName(username)) {
        return "Username is not correctly formatted; please ensure that your username contains
an underscore and is no more than five characters in length.";
    }
}

```

```

    }

    if (!checkPasswordComplexity(password)) {

        return "Password is not correctly formatted; please ensure that the password contains at
least eight characters, a capital letter, a number, and a special character.";

    }

    if (!checkCellPhoneNumber(cellNumber)) {

        return "Cell phone number incorrectly formatted or does not contain international code.";

    }


    // Successfully capture fields

    this.registeredUsername = username;

    this.registeredPassword = password;

    this.registeredCellNumber = cellNumber;


    return "Username successfully captured.\nPassword successfully captured.\nCell phone
number successfully added.";

}


// Verifies login credentials match registration data

public boolean loginUser(String username, String password) {

    if (this.registeredUsername == null || this.registeredPassword == null) {

        return false;

    }

    return this.registeredUsername.equals(username) &&
this.registeredPassword.equals(password);

}


// Returns dynamic login welcome or error feedback

public String returnLoginStatus(boolean isLoggedIn) {

    if (isLoggedIn) {

        return "Welcome " + this.firstName + ", " + this.lastName + " it is great to see you again.";

    } else {

```

```
        return "Username or password incorrect, please try again.";
    }
}
}
```

Message class

```
import java.util.Random;
```

```
public class MESSAGE {
```

```
    private String messageId;
```

```
    private int messageNumber;
```

```
    private String recipientCell;
```

```
    private String messageText;
```

```
    private String messageHash;
```

```
    private String status; // "Sent", "Stored", "Disregarded"
```

```
    // Constructor autogenerates the unique ID and numbers
```

```
    public MESSAGE(int messageCount, String recipientCell, String messageText) {
```

```
        this.messageNumber = messageCount;
```

```
        this.recipientCell = recipientCell;
```

```
        this.messageText = messageText;
```

```
        this.messageId = generateMessageID();
```

```
        this.messageHash = createMessageHash();
```

```
        this.status = "Disregarded"; // default
```

```
    }
```

```
    // Autogenerates a 10-digit random tracking number as a string
```

```
    private String generateMessageID() {
```

```
        Random rand = new Random();
```

```
        StringBuilder sb = new StringBuilder();
```

```
        for (int i = 0; i < 10; i++) {
```

```
            sb.append(rand.nextInt(10));
```

```
        }
```

```
        return sb.toString();
```

```
}
```

```
public boolean checkMessageID() {  
    return this.messageId != null && this.messageId.length() <= 10;  
}
```

```
// Confirms if the recipient follows length boundaries and starts with correct prefix codes  
public String checkRecipientCell() {  
    if (recipientCell.startsWith("+27") || recipientCell.startsWith("27") ||  
recipientCell.startsWith("0")) {  
        return "Cell phone number successfully captured.";  
    }  
    return "Cell phone number is incorrectly formatted or does not contain an international  
code. Please correct the number and try again.";  
}
```

```
// Generates uppercase text hash format:  
"FirstTwoDigits:MessageNumber:FIRSTWORDLASTWORD"  
public String createMessageHash() {  
    String firstTwoDigits = this.messageId.substring(0, 2);  
  
    // Clean and split message to extract words securely  
    String cleanedMsg = this.messageText.trim().replaceAll("[^a-zA-Z0-9 ]", "");  
    String[] words = cleanedMsg.split("\\s+");  
  
    String firstWord = words.length > 0 ? words[0] : "EMPTY";  
    String lastWord = words.length > 0 ? words[words.length - 1] : "EMPTY";  
  
    String hash = firstTwoDigits + ":" + this.messageNumber + ":" + firstWord + lastWord;  
    return hash.toUpperCase();  
}
```

```

// Verifies text size constraints and calculates overflow metrics if failed
public String validateMessageLength() {
    if (this.messageText.length() <= 250) {
        return "Message ready to send.";
    } else {
        int overflow = this.messageText.length() - 250;
        return "Message exceeds 250 characters by " + overflow + "; please reduce the size.";
    }
}

// Accessors and Mutators
public String getMessageId() { return messageId; }
public String getMessageHash() { return messageHash; }
public String getRecipientCell() { return recipientCell; }
public String getMessageText() { return messageText; }
public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

// Generates readable payload block for standard reporting
public String printMessageDetails() {
    return "Message ID: " + messageId +
        "\nMessage Hash: " + messageHash +
        "\nRecipient: " + recipientCell +
        "\nMessage: " + messageText +
        "\nStatus: " + status + "\n";
}
}

```

Quick chat app

```
import java.util.Scanner;

import java.util.ArrayList;

import javaapplication7.LOGIN;
```

```
public class QuickChatApp_1 {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        LOGIN authSystem = new LOGIN();

        System.out.println("--- Welcome to QuickChat: System Account Creation ---");

        System.out.print("Enter First Name: ");

        String fName = input.nextLine();

        System.out.print("Enter Last Name: ");

        String lName = input.nextLine();

        authSystem.setNames(fName, lName);

        // 1. Account Registration Loop (Part 1)

        boolean registered = false;

        while (!registered) {

            System.out.print("Create Username (Max 5 chars, must include '_'): ");

            String user = input.nextLine();

            System.out.print("Create Password (Min 8 chars, Upper, Number, Special): ");

            String pass = input.nextLine();

            System.out.print("Enter SA Cell Phone Number: ");

            String cell = input.nextLine();

            // Call the registration method from the Login class
```

```

String regResult = authSystem.registerUser(user, pass, cell);

System.out.println("\n" + regResult);


// If registration is successful, break the loop
if (regResult.contains("successfully added")) {
    registered = true;
} else {
    System.out.println("Registration failed due to formatting errors. Please try again.\n");
}
}

// 2. Authentication Login Loop (Part 1)
boolean loggedIn = false;
System.out.println("\n--- Access Portal: Sign-In ---");
while (!loggedIn) {
    System.out.print("Enter Username: ");
    String loginUser = input.nextLine();
    System.out.print("Enter Password: ");
    String loginPass = input.nextLine();

    // Verify credentials using the Login class
    loggedIn = authSystem.loginUser(loginUser, loginPass);
    System.out.println(authSystem.returnLoginStatus(loggedIn));

    if (!loggedIn) {
        System.out.println("Access Denied. Let's try that again.\n");
    }
}

// 3. Application Menu Loop (Part 2)
System.out.println("\nWelcome to QuickChat.");

```

```

System.out.print("How many messages would you like to handle this session?: ");

int maxMessages = input.nextInt();

input.nextLine(); // Clear scanner buffer window


ArrayList<MESSAGE> messageStore = new ArrayList<>();

int totalMessagesSentCount = 0;

int currentMessageCounter = 0;


boolean running = true;

while (running) {

    System.out.println("\n--- MAIN MENU ---");

    System.out.println("1) Send Messages");

    System.out.println("2) Show recently sent messages");

    System.out.println("3) Quit");

    System.out.print("Select an option (1-3): ");

    int choice = input.nextInt();

    input.nextLine(); // Clear scanner buffer


    switch (choice) {

        case 1:

            // Check if user exceeded the number of messages they defined at startup

            if (currentMessageCounter >= maxMessages) {

                System.out.println("You have reached your declared limit of " + maxMessages + "
messages for this session.");

                break;

            }


            System.out.print("\nEnter Recipient Cell Number: ");

            String recipient = input.nextLine();

            System.out.print("Enter Message text (Max 250 characters): ");

```

```

String text = input.nextLine();

// Instantiate a new Message object
MESSAGE currentMsg = new MESSAGE(currentMessageCounter, recipient, text);

// Validate message character limit length rules
String lengthCheck = currentMsg.validateMessageLength();
if (lengthCheck.contains("exceeds")) {
    System.out.println(lengthCheck);
    System.out.println("Message discarded due to excessive length.");
    break;
}

// Process dynamic options task choice selection menu
System.out.println("\nMessage Processing Options:");
System.out.println("-> Type 'Send' to dispatch");
System.out.println("-> Type 'Store' to process later");
System.out.println("-> Type '0' to disregard and delete");
System.out.print("Your choice: ");
String action = input.nextLine().trim();

if (action.equalsIgnoreCase("Send")) {
    currentMsg.setStatus("Sent");
    totalMessagesSentCount++;
    System.out.println("Message successfully sent");

    // Display full details immediately on screen after sending as required by brief
    System.out.println("\n--- Sent Dispatch Information ---");
    System.out.println(currentMsg.printMessageDetails());
} else if (action.equalsIgnoreCase("Store")) {
    currentMsg.setStatus("Stored");

```

```

        System.out.println("Message successfully stored");
    } else {
        currentMsg.setStatus("Disregarded");
        System.out.println("Press 0 to delete the message -> Message Deleted.");
    }

    messageStore.add(currentMsg);
    currentMessageCounter++;
    break;

case 2:
    // Required placeholder message output matching requirements
    System.out.println("\nComing Soon.");
    break;

case 3:
    running = false;
    // Accumulated totals displayed once program is exited
    System.out.println("\nTotal count of successfully dispatched messages: " +
totalMessagesSentCount);
    System.out.println("Thank you for using QuickChat. Goodbye!");
    break;

default:
    System.out.println("Invalid input choice. Please select 1, 2, or 3.");
}
}
input.close();
}
}

```

Bibliography

OpenAI. 2026. Chat-GPT (Version 5.2). [Large language model]. Available at: <https://chat.openai.com/> [Accessed: 01 June 2026].

Google. 2026. Gemini (Gemini 3 flash). [large language model]. Available at <https://gemini.google.com/> [Accessed 01 June 2026].

Goyvaerts, J. (2024) Regular-Expressions.info. [Online] Available at: <https://www.regular-expressions.info/> [Accessed: 11 April 2026].

J.Farrel. Java Programming. Cengage Learning. 2022

Oracle. Java Platform, standard edition documentation (Array list & file writer). Available at <https://docs.oracle.com/en/java/javase> [Accessed 21 April 2026]

ECMA International. (2017). The JSON data interchange syntax. Available at <https://www.json.org/json-en.html/> [Accessed 21 April 2026]

GITHUB REPOSITORY

[kekanakate24-a11y/PROG5121-POE](#)